

Multi-Tenancy in APIs

1. What is Multi-Tenancy? 🏢

Definition

Multi-tenancy is a software architecture pattern where a single instance of an application serves multiple tenants (customers/organizations) while keeping their data isolated and secure.

One service— many client/tenant

example: Multiple families live in one apartment building

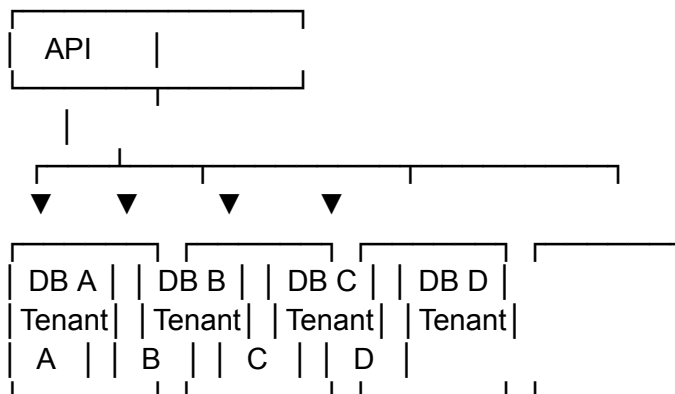
Benefit:

- Cost Reduction: One infrastructure serves all customers.
- Faster Deployment: Deploy once, all tenants get updates.
- Easier Maintenance: Single codebase to manage.

3. Types of Multi-Tenancy 🎯

3.1 Database per Tenant

Copy



Pros

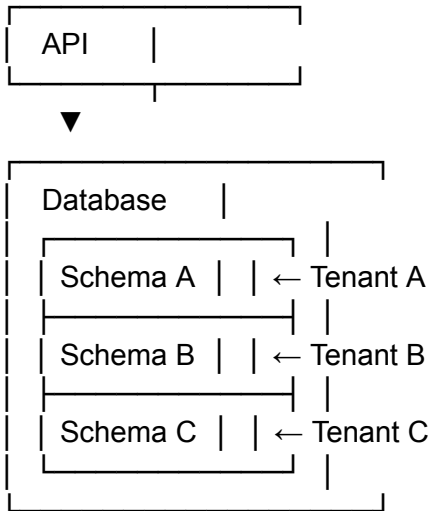
Cons

✓ Maximum data isolation	✗ Expensive (many databases)
✓ Easy to backup per tenant	✗ Hard to scale (many connections)
✓ Custom schema per tenant	✗ Complex migrations
✓ Easy to remove tenant	✗ Resource intensive

Best For: Enterprise clients, strict compliance (HIPAA, GDPR)

3.2 Shared Database, Separate Schemas

Copy

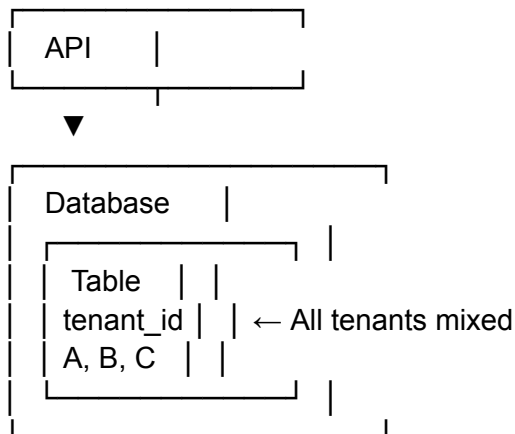


Pros	Cons
✓ Good data isolation	✗ Schema management complexity
✓ Lower cost than DB-per-tenant	✗ Limited database connections
✓ Easier backups	✗ Cross-tenant queries difficult

Best For: Medium-sized SaaS, 10-100 tenants

3.3 Shared Database, Shared Schema (Most Common)

Copy



Pros	Cons
✓ Lowest cost	✗ Risk of data leakage
✓ Easy to scale	✗ Complex queries (always filter tenant_id)
✓ Simple maintenance	✗ Hard to customize per tenant
✓ Easy cross-tenant analytics	✗ Noisy neighbor problem

Best For: Startups, high-volume SaaS (Slack, Zoom, etc.)

4. Database Strategies

Comparison Table

Strategy	Isolation	Cost	Complexity	Best For
DB per Tenant	★★★★★	\$\$\$\$\$	High	Enterprise
Schema per Tenant	★★★★	\$\$\$	Medium	Mid-market

Shared Schema	☆☆☆	\$	Low	Startups/SMB
---------------	-----	----	-----	--------------

Shared Schema Implementation Example

Copy

models/user.py

from sqlalchemy import Column, Integer, String, ForeignKey

class User(Base):

 __tablename__ = "users"

 id = Column(Integer, primary_key=True)

 tenant_id = Column(Integer, ForeignKey("tenants.id"), nullable=False) # ← KEY FIELD

 username = Column(String)

 email = Column(String)

Copy

Every query MUST filter by tenant_id

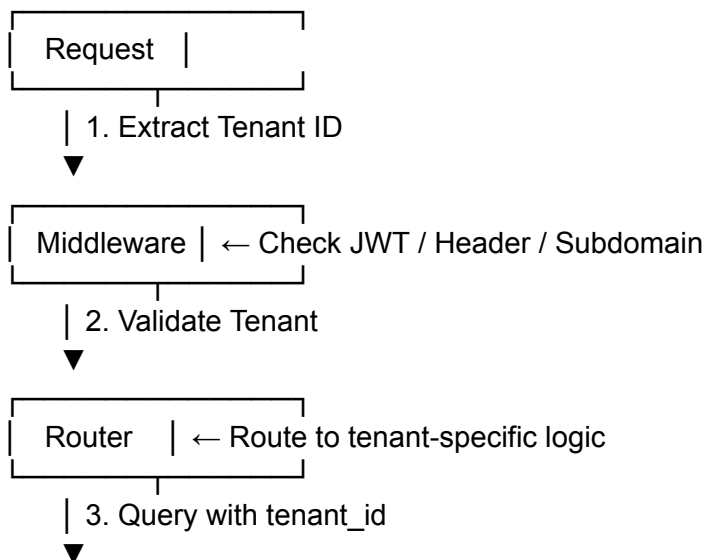
def get_users(db: Session, tenant_id: int):

 return db.query(User).filter(User.tenant_id == tenant_id).all()

5. Architecture Patterns 🏗️

5.1 Tenant Resolution Flow

Copy



5.2 Tenant Identification Methods

Method	Example	Pros	Cons
Subdomain	tenant1.app.com	Clean URLs	SSL complexity
Header	X-Tenant-ID: 123	Easy to implement	Can be spoofed
JWT Claim	{"tenant_id": 123}	Secure, stateless	Token size
Path	/api/tenant1/users	Simple	URL pollution

Recommended: JWT Claim + Header validation

6. Tenant Identification

Implementation Example (FastAPI)

Copy

```
# middleware/tenant_middlewre.py
from fastapi import Request, HTTPException
from jwt import decode_token

async def get_tenant_from_request(request: Request):
    # Method 1: From JWT token
    token = request.headers.get("Authorization")
    if token:
        payload = decode_token(token)
        tenant_id = payload.get("tenant_id")
        if tenant_id:
            return tenant_id

    # Method 2: From header
    tenant_id = request.headers.get("X-Tenant-ID")
    if tenant_id:
```

```
    return tenant_id

# Method 3: From subdomain
host = request.headers.get("host")
if host:
    subdomain = host.split(".")[0]
    if subdomain != "www":
        return get_tenant_by_subdomain(subdomain)

raise HTTPException(status_code=400, detail="Tenant ID required")
```

Dependency Injection (FastAPI)

Copy

```
# dependencies.py
from fastapi import Depends, HTTPException
from sqlalchemy.orm import Session

async def get_current_tenant(
    request: Request,
    db: Session = Depends(get_db)
) -> Tenant:
    tenant_id = await get_tenant_from_request(request)

    tenant = db.query(Tenant).filter(Tenant.id == tenant_id).first()
    if not tenant:
        raise HTTPException(status_code=404, detail="Tenant not found")

    if not tenant.is_active:
        raise HTTPException(status_code=403, detail="Tenant account suspended")

    return tenant
```

7. Security Considerations

Critical Security Rules

Rule	Why	How
Always filter by tenant_id	Prevent data leakage	Use query scopes

Validate tenant on every request	Prevent tenant switching	Middleware + JWT
Encrypt tenant data	Compliance requirements	Field-level encryption
Audit logs per tenant	Track suspicious activity	Separate log streams
Rate limit per tenant	Prevent abuse	Tenant-specific limits

Common Vulnerabilities

Vulnerability	Risk	Prevention
Tenant ID Manipulation	User accesses another tenant's data	Validate tenant from JWT, not user input
Missing tenant_id Filter	Query returns all tenants' data	Use ORM scopes, never raw queries
Cross-Tenant Caching	Cache leaks data between tenants	Include tenant_id in cache key
Shared Upload Storage	Files accessible across tenants	Separate folders per tenant

Secure Query Pattern

Copy

 WRONG - No tenant filter

```
def get_tasks(db: Session, task_id: int):
    return db.query(Task).filter(Task.id == task_id).first()
```

 CORRECT - Always filter by tenant

```
def get_tasks(db: Session, task_id: int, tenant_id: int):
    return db.query(Task).filter(
        Task.id == task_id,
        Task.tenant_id == tenant_id # ← CRITICAL
    ).first()
```

Types of Multi-Tenancy

There are three main types of multi-tenancy architecture, each with different levels of data isolation and cost.

Type 1: Database per Tenant

In this approach, each tenant gets their own separate database. The API remains the same, but when a request comes in, the system connects to that specific tenant's database.

Advantages:

- Maximum data isolation between tenants
- Easy to backup individual tenant data
- Can customize schema per tenant if needed
- Simple to remove a tenant (just delete their database)

Disadvantages:

- Most expensive option (many databases to maintain)
- Hard to scale as you need many database connections
- Complex migrations (must update all databases)
- Resource intensive for the server

Best For: Enterprise clients who need strict compliance like HIPAA or GDPR.

Type 2: Shared Database, Separate Schemas

In this approach, all tenants share one database, but each tenant has their own schema within that database. Think of it like one building with separate apartments.

Advantages:

- Good data isolation between tenants
- Lower cost than database per tenant
- Easier backups than multiple databases
- Better resource utilization

Disadvantages:

- Schema management becomes complex
- Limited database connections
- Cross-tenant queries are difficult
- Migrations still require careful planning

Best For: Medium-sized SaaS companies with 10 to 100 tenants.

Type 3: Shared Database, Shared Schema (Most Common)

In this approach, all tenants share the same database and the same schema. Every table has a `tenant_id` column to identify which tenant the data belongs to.

Advantages:

- Lowest cost (one database for all)
- Easy to scale
- Simple maintenance
- Easy cross-tenant analytics

Disadvantages:

- Higher risk of data leakage if not implemented correctly
- Every query must filter by `tenant_id`
- Hard to customize per tenant
- Noisy neighbor problem (one tenant can affect others)

Best For: Startups and high-volume SaaS products like Slack, Zoom, and similar platforms.

4. Database Strategies

When choosing a database strategy for multi-tenancy, you need to balance isolation, cost, and complexity.

Database per Tenant gives you the highest isolation but costs the most. You will need to manage many database connections and run migrations on all databases whenever you update the schema.

Schema per Tenant gives you medium isolation at medium cost. You still need to manage multiple schemas, but they all live in one database instance.

Shared Schema gives you the lowest isolation but also the lowest cost. All tenant data lives in the same tables, separated only by a `tenant_id` column. This is the most common choice for startups.

Key Implementation Point: In shared schema, every single query **MUST** filter by `tenant_id`. If you forget this filter even once, you risk leaking data between tenants.

5. Architecture Patterns

Tenant Resolution Flow

When a request comes to your API, it needs to go through several steps before accessing data.

First, the system extracts the tenant ID from the request. This can come from a JWT token, a header, a subdomain, or a URL path.

Second, the middleware validates that the tenant exists and is active. If the tenant is not found or is suspended, the request is rejected.

Third, the router processes the request with the tenant context. All database queries automatically include the `tenant_id` filter.

Finally, the database returns only data belonging to that tenant.

Tenant Identification Methods

There are several ways to identify which tenant a request belongs to.

Subdomain: Each tenant gets their own subdomain like `tenant1.app.com` or `tenant2.app.com`. This is clean and professional but requires SSL certificate management for each subdomain.

Header: The client sends a custom header like `X-Tenant-ID: 123` with every request. This is easy to implement but can be spoofed if not properly secured.

JWT Claim: The tenant ID is embedded in the JWT token when the user logs in. This is secure and stateless because the token is signed and cannot be tampered with.

Path: The tenant ID is part of the URL like `/api/tenant1/users`. This is simple but makes URLs messy and harder to manage.

Recommended Approach: Use JWT claims combined with header validation for the best security and ease of implementation.

6. Tenant Identification

Implementation in FastAPI

You need to create a middleware or dependency that extracts the tenant ID from every incoming request.

First, check the JWT token for a `tenant_id` claim. This is the most secure method because the token is signed and cannot be modified by the client.

Second, if no JWT is present, check for a custom header like `X-Tenant-ID`. This is useful for server-to-server communication.

Third, you can also check the subdomain from the host header if you are using subdomain-based tenancy.

If none of these methods return a valid tenant ID, reject the request with a 400 error.

Dependency Injection

In FastAPI, you can create a dependency function that automatically validates the tenant on every endpoint.

The function extracts the tenant ID from the request, queries the database to find the tenant, and checks if the tenant is active.

If the tenant is not found, return a 404 error.

If the tenant is suspended or inactive, return a 403 error.

If everything is valid, return the tenant object so it can be used in the endpoint logic.

This dependency can then be added to any route that requires tenant validation.

7. Security Considerations

Critical Security Rules

Always filter by `tenant_id`: Every single database query must include a `tenant_id` filter.

Never trust that the ORM will do this automatically.

Validate tenant on every request: Do not assume the tenant is valid just because it was valid earlier. Check on every single request.

Encrypt tenant data: If you are handling sensitive data, consider encrypting it at rest.

This is especially important for compliance requirements.

Audit logs per tenant: Keep separate audit logs for each tenant so you can track suspicious activity and comply with regulations.

Rate limit per tenant: Implement rate limiting on a per-tenant basis to prevent one tenant from abusing the system and affecting others.

Common Vulnerabilities

Tenant ID Manipulation: A user might try to change the `tenant_id` in their request to access another tenant's data. Prevent this by always extracting `tenant_id` from the JWT token, never from user input.

Missing `tenant_id` Filter: A developer might forget to add the `tenant_id` filter to a query. Prevent this by using ORM scopes or query helpers that automatically add the filter.

Cross-Tenant Caching: If you use caching, make sure the cache key includes the `tenant_id`. Otherwise, one tenant might see another tenant's cached data.

Shared Upload Storage: If you store files, keep them in separate folders per tenant. Do not allow tenants to access files from other tenants.

8. Pros and Cons

Advantages of Multi-Tenancy

Cost Efficiency: You can reduce infrastructure costs by 60 to 80 percent because you are sharing resources across all tenants.

Easier Maintenance: You only have one codebase and one deployment pipeline to manage.

Faster Feature Rollout: When you deploy a new feature, all tenants get it instantly. No need to update each tenant separately.

Better Resource Utilization: Shared compute and storage means you can serve more tenants with fewer resources.

Unified Analytics: You can analyze data across all tenants to understand usage patterns and improve your product.

Disadvantages of Multi-Tenancy

Data Leakage Risk: If not implemented correctly, one tenant might see another tenant's data. This is the biggest risk and must be prevented with strict filtering.

Noisy Neighbor: One tenant might use so many resources that it affects the performance for other tenants. Prevent this with rate limiting and resource quotas.

Customization Limits: It is harder to customize the application for individual tenants because everyone shares the same codebase.

Complex Migrations: When you update the database schema, you need to be careful not to break anything for any tenant.

Single Point of Failure: If your single instance goes down, all tenants are affected. Prevent this with redundancy and backups.

9. Implementation Checklist

Phase 1: Foundation

Create a tenants table in your database with fields like id, name, subdomain, is_active, and created_at.

Add a tenant_id column to all existing tables that store tenant-specific data.

Create a middleware or dependency that extracts and validates the tenant ID from every request.

Update your JWT token generation to include the tenant_id claim.

Phase 2: Security

Add tenant validation to every endpoint. No endpoint should be accessible without a valid tenant.

Implement query scopes or helper functions that automatically add the tenant_id filter to all queries.

Add audit logging that records all actions per tenant.

Set up rate limiting that applies per tenant, not globally.

Phase 3: Operations

Create a tenant onboarding flow that allows new tenants to sign up and get their own tenant ID.

Create a tenant deletion flow that properly cleans up all data when a tenant leaves.

Set up a backup strategy that can restore individual tenants if needed.

Create a monitoring dashboard that shows metrics per tenant.

Phase 4: Optimization

Implement caching that includes `tenant_id` in the cache key.

Set up database connection pooling to handle many tenants efficiently.

Add performance monitoring to detect slow queries or resource issues.

Run load tests to ensure your system can handle many tenants at once.

10. Real-World Examples

Slack uses a shared schema approach because they have a high volume of small tenants. They prioritize scale over isolation.

Shopify uses a schema per tenant approach because they need medium isolation and customization for their merchants.

Salesforce uses a database per tenant approach for their enterprise clients who need strict compliance and data isolation.

Zoom uses a shared schema approach because they need to scale to millions of users efficiently.

HubSpot uses a hybrid approach where different pricing tiers get different multi-tenancy strategies.

11. My Implementation Plan

Week 1: Setup

Days 1 and 2: Create the tenant model and run the database migration.

Days 3 and 4: Add `tenant_id` column to all existing tables like users, projects, tasks, and organizations.

Day 5: Create the tenant identification middleware that extracts tenant ID from JWT or headers.

Week 2: Security

Days 1 and 2: Update JWT token generation to include `tenant_id` claim.

Days 3 and 4: Add tenant validation dependency to all existing endpoints.

Day 5: Implement query scopes that automatically filter by `tenant_id`.

Week 3: Testing

Days 1 and 2: Write tests to verify tenant isolation. Try to access one tenant's data from another tenant's context.

Days 3 and 4: Run a security audit to find any potential data leakage points.

Day 5: Run performance tests to ensure the system can handle multiple tenants.

Week 4: Documentation

Days 1 and 2: Update API documentation to include tenant-related endpoints and requirements.

Days 3 and 4: Write a tenant onboarding guide for new customers.

Day 5: Final review and deployment to production.

Key Learnings Summary

Multi-Tenancy Definition: A single application serving multiple customers while keeping their data isolated.

Database Strategies: Three options: database per tenant, schema per tenant, or shared schema.

Tenant Identification: Use JWT claims, headers, subdomains, or URL paths to identify tenants.

Security Risks: Data leakage, tenant switching, cross-tenant caching, and shared storage.

Implementation Steps: Create tenant model, add tenant_id to tables, create middleware, validate on every request.

Document Created: March 17, 2026

Status: Learning Complete

Next Step: Begin Implementation

This document covers everything I learned about multi-tenancy today. I now understand the concepts, strategies, security considerations, and implementation steps. Ready to start building! 🚀